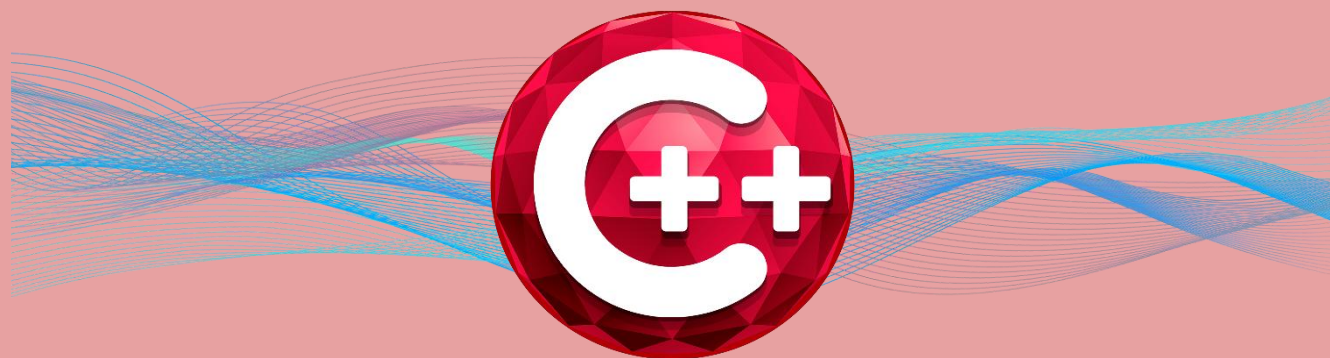




RAD Studio | C++Builder 13

FLORENCE



MÓDULO 1 - BÁSICO



FIREDAC

FIREDAC
ACESSO A DADOS

C++

C++ BUILDER



Criado por: Thiago Santos

MVP Embarcadero. 2026

SUMÁRIO

Sumário	2
Introdução ao Curso de C++Builder 13 Florence.....	3
O que você vai aprender neste curso.....	4
Sobre o instrutor.....	5
FireDAC: Visão Geral.....	7
Data Access Engine.....	7
API Unificada.....	7
Alto Desempenho	8
Arquitetura do Firedac	8
Componentes não visuais	9
Componentes visíveis [GUIx].....	10
Drivers nativos [Phys].....	10
Configurando conexões	11
Usando o utilitário FDExplorer com C++Builder	11
Usando Editor do FDConnection C++Builder	13
Criando uma aplicação conectada ao Banco de dados com C++Builder	15
Estabelecendo a conexão com o banco de dados	15
Seleção de linhas do banco de dados usando C++BUILDER.....	17
Preparando o aplicativo para o tempo de execução C++Builder.....	18
Definindo Conexões em tempo de execução	18
Arquivo de definição de conexão	19
Criação de uma definição de conexão persistente	21
Criando uma definição de conexão privada	22
Criação de uma definição de conexão temporária	24
Editando uma definição de conexão.....	25

Seja bem-vindo!!

Introdução ao Curso de C++Builder 13 Florence

Este curso foi desenvolvido para apresentar o **C++Builder 13 Florence**, a versão mais recente da plataforma de desenvolvimento da Embarcadero, com foco em quem deseja aprender **C++ de forma prática, produtiva e aplicada ao desenvolvimento de aplicações reais**.

Ao longo desta formação, o aluno será introduzido aos principais conceitos do **C++ funcional aplicado ao C++Builder**, entendendo não apenas a linguagem, mas também como utilizá-la de forma eficiente dentro de um ambiente RAD (Rapid Application Development), que acelera o processo de criação de software sem *abrir mão de desempenho e controle*.

A importância de utilizar o C++Builder 13 Florence

Utilizar a versão mais recente do C++Builder é fundamental para garantir **compatibilidade com tecnologias atuais**, maior estabilidade da IDE e acesso aos recursos mais modernos da linguagem C++. O C++Builder 13 Florence traz melhorias significativas no compilador baseado em Clang, suporte aprimorado ao padrão moderno do C++, além de uma IDE mais rápida, segura e alinhada às necessidades do mercado atual.

Aprender com uma versão atual evita retrabalho futuro, reduz problemas de compatibilidade e prepara o aluno para atuar em projetos profissionais, corporativos e comerciais, utilizando ferramentas reconhecidas e amplamente adotadas por empresas.

O que você vai aprender neste curso

Este curso tem como objetivo fornecer uma **base funcional e prática**, permitindo que o aluno desenvolva aplicações reais desde os primeiros módulos. Entre os principais conhecimentos adquiridos, destacam-se:

- Fundamentos do C++ aplicados ao C++Builder
- Estrutura da IDE e fluxo de desenvolvimento
- Criação de aplicações visuais utilizando VCL
- Manipulação de eventos e componentes
- Lógica de programação aplicada a projetos reais
- Integração básica com banco de dados
- Boas práticas para desenvolvimento em C++Builder

Ao final do curso, o aluno terá conhecimento suficiente para **criar aplicações funcionais**, entender projetos existentes e evoluir para níveis mais avançados da linguagem e da ferramenta.

Por que escolher o C++Builder

O C++Builder se diferencia por unir o **alto desempenho do C++** com a **produtividade do desenvolvimento visual**. Ele permite criar aplicações robustas, rápidas e profissionais com muito menos código do que abordagens tradicionais, mantendo total controle sobre a linguagem e o desempenho.

Além disso, o C++Builder é amplamente utilizado em sistemas corporativos, aplicações industriais, softwares comerciais e soluções que exigem **performance, estabilidade e longevidade**. Aprender C++Builder é investir em uma tecnologia madura, sólida e ainda extremamente relevante no mercado.

Este curso é o primeiro passo para quem deseja dominar o **C++ moderno aplicado ao desenvolvimento profissional**, utilizando uma das ferramentas mais completas e produtivas disponíveis atualmente.

Está pronto? Vamos começar!!

Sobre o instrutor

Meu nome é **Thiago Santos** e sou profissional da área de tecnologia, atuando há vários anos com **desenvolvimento de software, análise de negócios e treinamentos técnicos**, com foco na linguagem **C++** e na plataforma **C++Builder**.

Ao longo da minha trajetória profissional, tive a oportunidade de trabalhar diretamente com projetos reais, ambientes corporativos e soluções que exigem **alto desempenho, estabilidade e manutenção a longo prazo**. Essa experiência prática me permitiu entender não apenas a linguagem C++, mas também como aplicá-la de forma eficiente no dia a dia de empresas e equipes de desenvolvimento.

Sou **MVP da Embarcadero na linguagem C++**, reconhecimento concedido a profissionais que contribuem ativamente para a comunidade, compartilham conhecimento e utilizam as tecnologias Embarcadero em projetos reais. Além disso, atuo como **instrutor, agente de negócios e consultor técnico**, auxiliando empresas e desenvolvedores na adoção e evolução do C++Builder em seus sistemas.

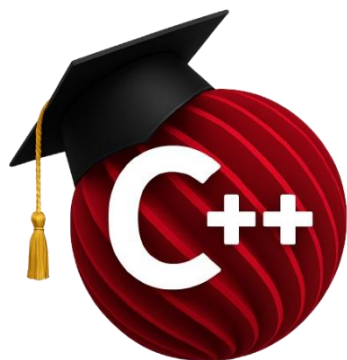
Neste curso, meu objetivo não é apenas ensinar comandos ou conceitos isolados, mas **transmitir a forma correta de pensar, estruturar e desenvolver aplicações em C++Builder**, com base em experiências reais de mercado. Todo o conteúdo foi planejado para ser direto, prático e aplicável, evitando excesso de teoria desconectada da prática.

Acredito que aprender programação vai muito além de escrever código: envolve entender processos, tomar decisões técnicas corretas e criar soluções que realmente funcionem. É exatamente essa visão que compartilho ao longo deste curso.

Seja bem-vindo(a) à formação. Espero que este conteúdo contribua de forma significativa para sua evolução profissional e técnica no desenvolvimento com **C++Builder**.

Meus contatos e plataformas estão disponíveis:

<https://sam-cpp.wordpress.com/>



ACESSO A DADOS COM FIREDAC

PERFIL

Este treinamento acadêmico visa apresentar ao aluno um ambiente integrado de desenvolvimento de alta produtividade, que permita a criação de aplicações de todos os tipos e para as mais importantes plataformas da atualidade.

Neste capítulo serão apresentados os conceitos e componentes envolvidos no acesso a dados e componentes de manipulação dos mesmos.

Caso queira se aprofundar e se especializar no conhecimento da ferramenta, sua linguagem, tecnologias e recursos, uma lista de centros de treinamentos oficiais está disponível:

www.embarcadero.com.br

CONTATO

academico@embarcadero.com.br

O que você verá:

Este módulo do treinamento acadêmico abordará:

- Arquitetura do FireDAC
- Configuração de Conexões
- FDExplorer e FDConnection
- Aplicação conectada a banco de dados com FireDAC
- Conexão em tempo de execução

HABILIDADES

FIREDAC: VISÃO GERAL



O FireDAC é uma biblioteca Universal Data Access para o desenvolvimento de aplicativos para vários dispositivos, conectados a bancos de dados corporativos que foi incorporada ao RAD Studio (Delphi e C++ Builder) a partir da versão XE3. Com sua poderosa arquitetura universal, FireDAC permite acesso direto de alta velocidade nativa de Delphi e C++ Builder para InterBase, SQLite, MySQL, SQL Server, Oracle, PostgreSQL, DB2, SQL Anywhere, Advantage DB, Firebird, Access, Informix, DataSnap e mais, incluindo o banco de dados NoSQL MongoDB.

Trata-se de um mecanismo de acesso poderoso, mas fácil de usar, que oferece suporte, abstrai e simplifica o acesso aos dados, fornecendo todos os recursos necessários para construir aplicativos corporativos de alta performance do mundo real. FireDAC fornece uma API comum para acessar diferentes back-ends de banco de dados, sem abrir mão de recursos exclusivos e específicos do banco de dados e sem comprometer o desempenho. Use FireDAC em aplicações Android, iOS, Windows, Mac OS X e Linux que você está desenvolvendo para PCs, tablets e smartphones.

DATA ACCESS ENGINE

Os conjuntos de dados FireDAC são construídos em cima de um poderoso mecanismo de acesso a dados. Este mecanismo leve, eficaz e flexível pode ser usado diretamente em aplicativos e serve como uma base poderosa para a API de conjuntos de dados. O mecanismo consiste nos componentes do conjunto de dados e nas camadas não componentes, representados pelas APIs orientadas a objetos flexíveis.

Classes descendentes de TDataSet fáceis de usar, incluindo TFDQuery, TFDMemTable, TFDStoredProc e TFDTable. Classes de conjunto de dados que são altamente compatíveis com conjuntos de dados BDE originais e ClientDataSet. Um dos conjuntos de dados in-memory mais rápidos, com classificação, filtragem, agregações, índices filtrados e de expressão, persistência e muito mais. Mecanismo SQL local para executar consultas SQL em conjuntos de dados.

API UNIFICADA

O FireDAC fornece uma API Unificada com uma gama de recursos que ajudam a abstrair as diferenças entre os sistemas de banco de dados, tornando mais fácil escrever código que não precise se preocupar com os diferentes dialetos do DBMS ou outras diferenças sutis entre os DBMS's *.

DBMS - Database Management System (Sistema de Gerenciamento de Banco de Dados)

Em consequência dessa API unificada alguns recursos poderosos são benefícios do FireDAC:

- Abstração do dialeto SQL por meio de sequências de escape FireDAC, instruções condicionais e macros
- Unificação de tipo de dados com mapeamento de tipo de dados flexível e ajustável

- Relatórios de erros unificados, incluindo informações de erros independentes e específicas do DBMS
- Suporte a transações unificadas, com transações separadas de leitura e atualização, e acesso a todo o poder do suporte de transações específicas do InterBase e Firebird
- Suporte para várias codificações Unicode e ANSI
- Recuperação automática de conexão, restabelecendo automaticamente a conexão em caso de ambiente instável
- Modo de conexão desconectado, permitindo que o aplicativo continue a funcionar sem uma conexão física com um banco de dados
- Suporte a eventos de banco de dados e notificações
- Suporte a scripts SQL unificados
- Recursos estendidos de recuperação de metadados

ALTO DESEMPENHO

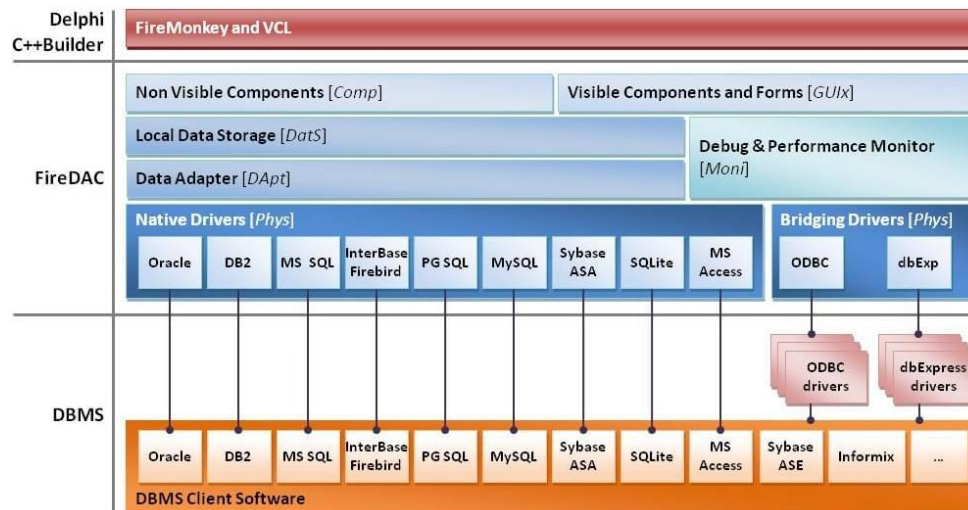
O acesso ao banco de dados é otimizado usando muitas técnicas diferentes, geralmente encontradas apenas em componentes específicos do banco de dados, o que permite obter acesso aos dados mais rápido pronto para uso. São algumas dessas técnicas:

- O modo Live Data Window permitindo uma navegação bidirecional rápida por meio de grandes conjuntos de dados
- Execução de comandos Array Data Manipulation Language (DML) e Command Batches para aplicativos em lote e para minimizar o tráfego de rede
- Busca personalizável e flexível de "conjunto de linhas"
- Suporte direto para execução de comando assíncrona, tempo limite de execução de comando e cancelamento de execução de comando
- Geração e execução de comando de atualização automática, eficiente e sofisticada
- Modo de atualizações em cache com capacidade de rastrear alterações correlacionadas para vários conjuntos de dados com atualizações em cascata
- Suporte completo para campos de incremento automático, incluindo aqueles baseados em geradores (generators) e acionadores(triggers) de tabela.

ARQUITETURA DO FIREDAC

FireDAC tem uma arquitetura flexível, poderosa e extensível.

Trata-se de uma arquitetura de múltiplas camadas fracamente acoplada, onde as camadas fornecem serviços. Uma API de serviço é definida como uma interface COM que outras camadas podem solicitar usando a fábrica de interface.



Quando uma implementação de interface não é encontrada, uma exceção é levantada. Para vincular a implementação a um aplicativo, a unidade correspondente também deve estar vinculada. Essas implementações podem ser obrigatórias ou opcionais e você pode escolher entre implementações alternativas.

Por exemplo, a interface `IFDGUIxWaitCursor` define a API para o cursor de espera do mouse (ampulheta). Ele tem três implementações alternativas (provedores):

`FireDAC.VCLUI.Wait` - esta unidade contém a implementação de aplicativos VCL GUI.

`FireDAC.FMXUI.Wait` - esta unidade contém a implementação de aplicativos FireMonkey GUI.

`FireDAC.ConsoleUI.Wait` - esta unidade contém a implementação de aplicativos de console.

A implementação de uma GUI ou cursor de espera do mouse do console é obrigatória e deve estar sempre vinculada ao aplicativo. Caso contrário, a seguinte exceção é gerada:

Object factory for class {3E9B315B-F456-4175-A864-B2573C4A2201} missing.
To register it, you can drop component [TFDGUIxWaitCursor] into your project

Observação: a mensagem de exceção sugere a unidade que deve ser incluída em seu projeto para vincular a implementação da interface padrão.

COMPONENTES NÃO VISUAIS

A camada Nonvisible Components [Comp] representa as interfaces públicas FireDAC como componentes não visuais C++Builder, semelhantes a outros componentes de acesso a dados C++Builder. Inclui componentes como `TFDConnection` (estabelece a conexão), `TFDQuery` (executa a consulta), `TFDStoredProc` (executa o procedimento armazenado), `TFDMemTable` (conjunto de dados na memória) e assim por diante.

TIPO DO COMPONENTE	FIREDAC
CONEXÃO	 TFDConnection
DATASET	 TFDMemTable
	 TFDQuery
	 TFDStoredProc
	 TFDTable

As unidades principais são:

- FireDAC.Comp.DataSet
- FireDAC.Comp.Client
- FireDAC.Comp.Script

COMPONENTES VISÍVEIS [GUIX]

A camada Visible Components [GUIx] fornece uma maneira de interagir com o usuário final a partir de um aplicativo FireDAC. É um conjunto de componentes de alto nível que permite adicionar os diálogos do usuário final para as operações de banco de dados padrão, como uma operação de Login ou Espera. Inclui componentes como TFDGUIxWaitCursor (cursor de espera), TFDGUIxLoginDialog (caixa de diálogo de login), TFDGUIxErrorDialog (caixa de diálogo de erro) e assim por diante. A camada fornece implementações para plataformas VCL (VCL), FireMonkey (FMX) e Console (Console).

As unidades principais são:

- FireDAC.UI.Intf
- FireDAC. <plataforma> UI.Xxxx
- FireDAC.Comp.UI

DRIVERS NATIVOS [PHYS]

Os Drivers Nativos [Phys] implementam o acesso a um SGBD usando uma API de baixo nível de alto desempenho, que é recomendada pelo fornecedor do SGBD. Eles adaptam precisamente os recursos específicos do DBMS à API FireDAC. Todos os drivers nativos foram testados e otimizados para um DBMS. Eles incluem TFDPhys <DBMS> DriverLink, bem como componentes de serviço.

As unidades principais são:

- FireDAC.Phys. <DBMS> Wrapper
- FireDAC.Phys. <DBMS> Meta
- FireDAC.Phys. <DBMS>:
 - FireDAC.Phys.ADS
 - FireDAC.Phys.ASA
 - FireDAC.Phys.DS

- FireDAC.Phys.DB2
- FireDAC.Phys.IB
- FireDAC.Phys.Infx
- FireDAC.Phys.FB
- FireDAC.Phys.MSAcc
- FireDAC.Phys.MSSQL
- FireDAC.Phys.MySQL
- FireDAC.Phys.Oracle
- FireDAC.Phys.PG
- FireDAC.Phys.SQLite
- FireDAC.Phys.TData
- FireDAC.Phys.MongoDB

CONFIGURANDO CONEXÕES

Os componentes FireDAC utilizam o conceito de definições de conexão para enviar todos os parâmetros necessários para estabelecer uma conexão com o banco de dados, como servidor, banco de dados, User_Name ao nível do driver FireDAC (em tempo de execução e tempo de design).

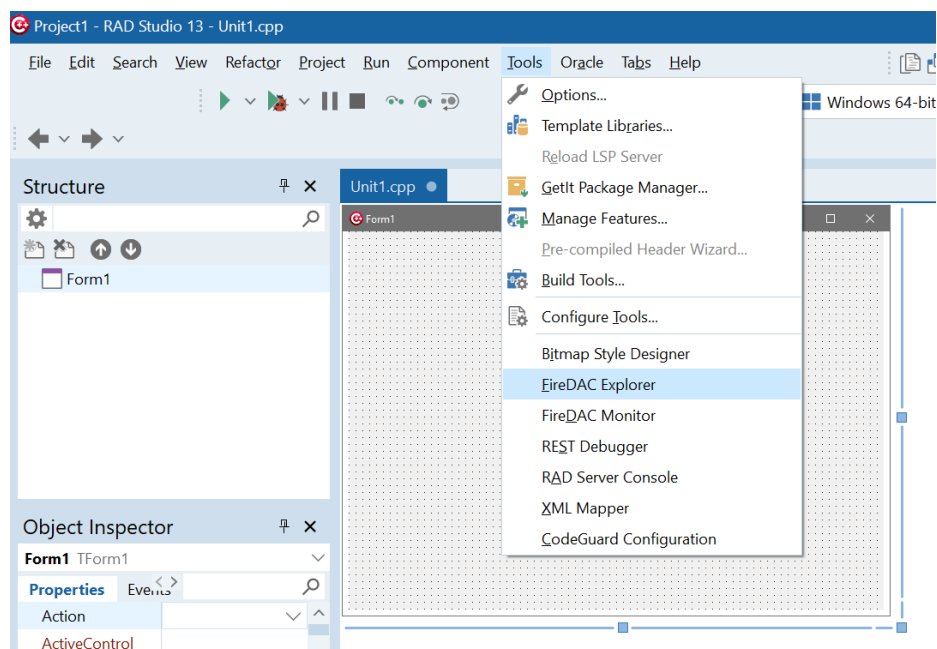
Uma definição de conexão é um conjunto de parâmetros que define como conectar um aplicativo a um DBMS usando um driver FireDAC específico. É o equivalente a um alias BDE, ADO UDL (string de conexão OLEDB armazenada) ou ODBC Data Source Name (DSN).

Basicamente o FireDAC fornece 2 formas de estabelecer a conexão com os DBMSs em tempo de design através de definição de conexão: utilizando o FDEplorer ou utilizando uma definição temporária através do editor de tempo de design do FDConnection.

USANDO O UTILITÁRIO FDEXPLORER COM C++BUILDER

O utilitário FDEplorer é a principal ferramenta para manter as definições de conexão persistentes centralizadas. Leia a referência do FDEplorer para entender o uso detalhado desta ferramenta.

Para executar o FDEplorer, clique em *Ferramentas* → *FireDAC* → *Explorer* no IDE.

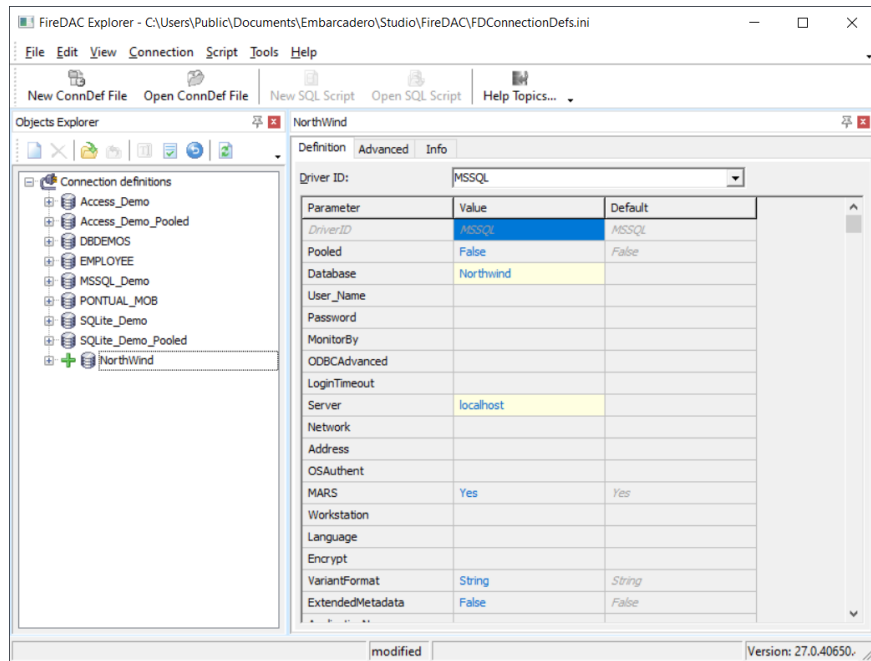


Em seguida, clique em Ctrl-N para criar uma nova definição de conexão vazia.

O valor do parâmetro DriverID especifica o driver que você decide usar. Depois de definir o DriverID como MSSQL, o FireDAC exibe o conjunto de parâmetros específico do driver. Para o Microsoft SQL Server, os parâmetros específicos do driver incluem:

Parâmetro	Descrição
Servidor	O identificador do servidor SQL Server. Se o host tiver apenas um único servidor padrão, esse valor será o endereço do host.
Base de dados	O nome do banco de dados padrão.
OSAuthent	Se estiver definido como Sim , o FireDAC usará a autenticação do Windows. Se estiver definido como Não (por padrão), a autenticação do MS SQL Server será usada.
Nome do usuário	O nome de usuário de login, se OSAuthent = Não.
Senha	A senha de login, se OSAuthent = Não.
MetaDefSchema	Nome do esquema padrão. O código de tempo de design exclui um nome de esquema de um nome de objeto, se for igual a MetaDefSchema.

A próxima captura de tela mostra a configuração da definição da conexão:



Pressione Ctrl - A para salvar a definição de conexão no arquivo de definição de conexão. Para testar uma nova definição de conexão, clique no sinal "+" no item da árvore. O explorador então mostra a caixa de diálogo de login. Após um login bem-sucedido, o nó da árvore se expande e permite que você faça uma busca detalhada nos objetos do banco de dados.

Nota: Se você adicionar uma nova definição de conexão persistente usando FDE Explorer enquanto o IDE C++Builder está em execução, a conexão não é visível para o código de tempo de design FireDAC. Para atualizar a lista de definição de conexão persistente, você precisa reiniciar o C++Builder IDE.

Agora a definição de conexão está pronta para uso no C++Builder. Basta definir o valor da propriedade `TFDConnection.ConnectionDefName` com o nome da definição de conexão recém-criada.

USANDO EDITOR DO FDConnection C++BUILDER

O editor de tempo de design do componente `TFDConnection` é o ambiente usado para manter os parâmetros de conexão temporários. Clique duas vezes em qualquer componente `TFDConnection` em tempo de design. O pacote FireDAC exibe a caixa de diálogo Editor de conexão:



O Editor do FDConnection oferece uma funcionalidade semelhante ao FDEplorer. Novamente, você deve começar definindo o seguinte:

- Driver ID - para criar uma definição de conexão temporária do zero (nosso caso);
- Nome da definição de conexão - para criar uma conexão temporária que substitui os parâmetros de uma conexão persistente existente.

Novamente, você precisa preencher os parâmetros conforme especificado no capítulo acima. Esta caixa de diálogo oferece as seguintes funções:

- O botão Testar - para testar a definição da conexão.
- O botão Assistente - para chamar um assistente de definição de conexão específico do DBMS, se disponível.
- O botão Reverter para o padrão - para redefinir os parâmetros para seus valores padrão.
- O botão Ajuda - para ir para uma página de ajuda com uma descrição dos parâmetros atuais do driver.

- A página de informações - para tentar se conectar a um DBMS e obter informações sobre a conexão.
- A página SQL Script - para executar os comandos de script SQL nesta conexão.

Após pressionar o botão OK do editor, FireDAC carrega os parâmetros de conexão na propriedade `TFDConnection.Params` e define a propriedade `TFDConnection.DriverName` com o valor escolhido. Depois de atribuir um nome de definição de conexão persistente à propriedade `TFDConnection.ConnectionDefName` ou preencher os parâmetros de definição de conexão temporária na propriedade `TFDConnection.Params`, defina a propriedade `TFDConnection.Connected` como `True`. Se os parâmetros forem especificados corretamente a conexão é estabelecida.

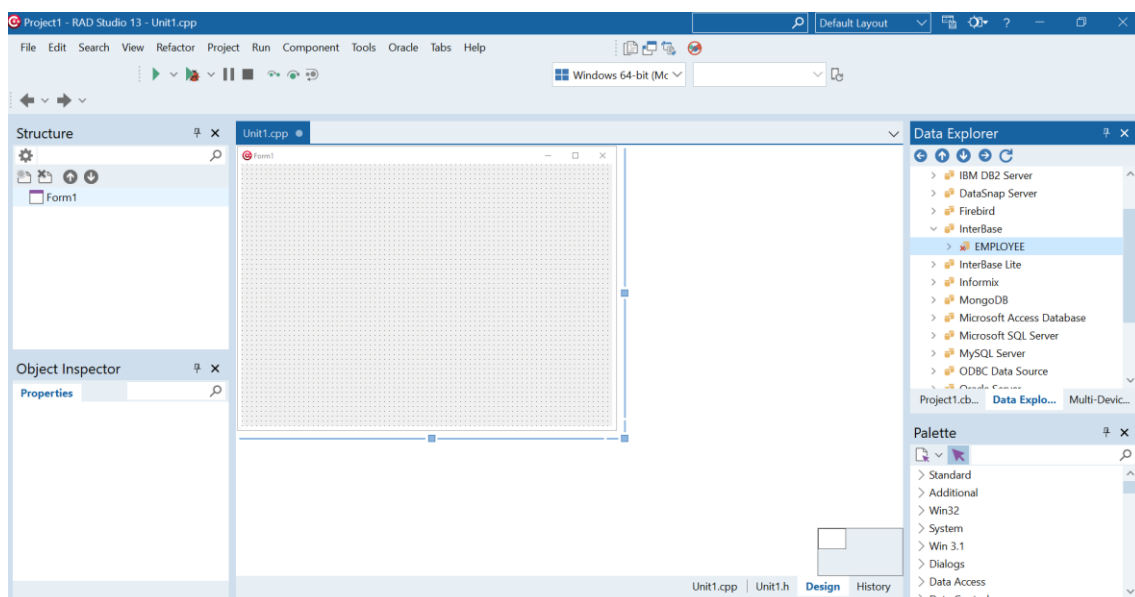
CRIANDO UMA APLICAÇÃO CONECTADA AO BANCO DE DADOS COM C++BUILDER

Criaremos uma primeira aplicação de conectada a um banco de dados através do FireDAC dividindo sua construção em 3 etapas principais:

- Estabelecendo a conexão com o banco de dados: como usar C++Builder para criar uma aplicação que se conectará a um banco de dados.
- Selecionando linhas do banco de dados: conecte os dados a uma grade e exiba-os em tempo de design.
- Preparando o aplicativo para o tempo de execução: descreve as etapas necessárias para fazer um aplicativo funcionar como um executável autônomo (tempo de execução).

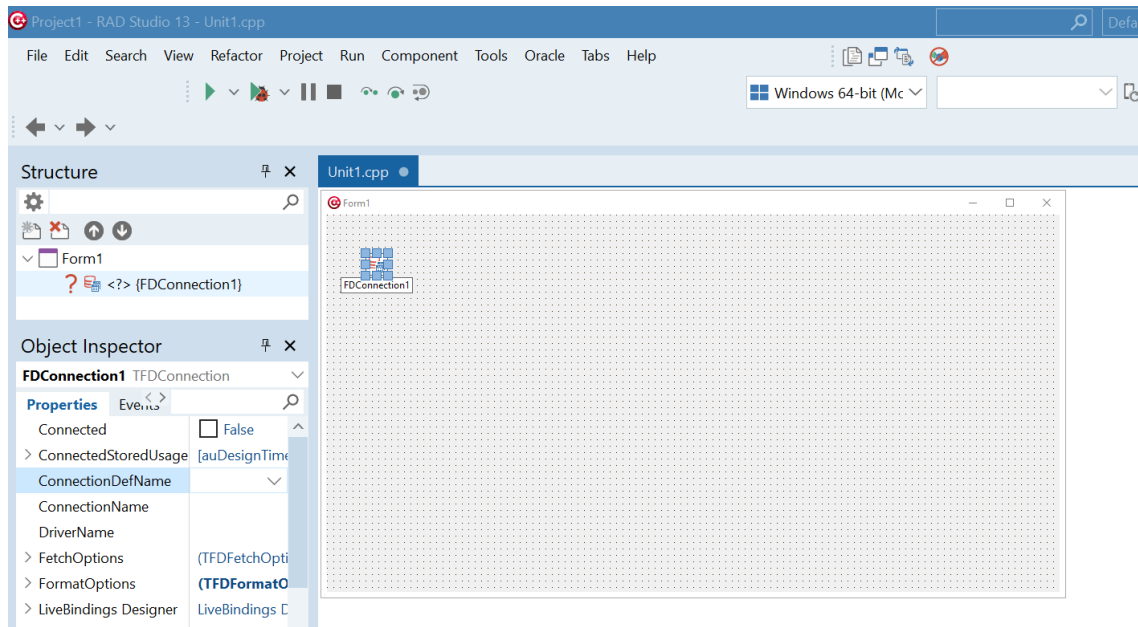
ESTABELECENDO A CONEXÃO COM O BANCO DE DADOS

Neste momento, utilizaremos o banco de dados InterBase, que acompanha as edições do C++Builder 13 e é gratuito para utilização em desenvolvimento (Developer Edition). Para estabelecermos a conexão, usaremos uma configuração de acesso predefinida: `EMPLOYEE`.



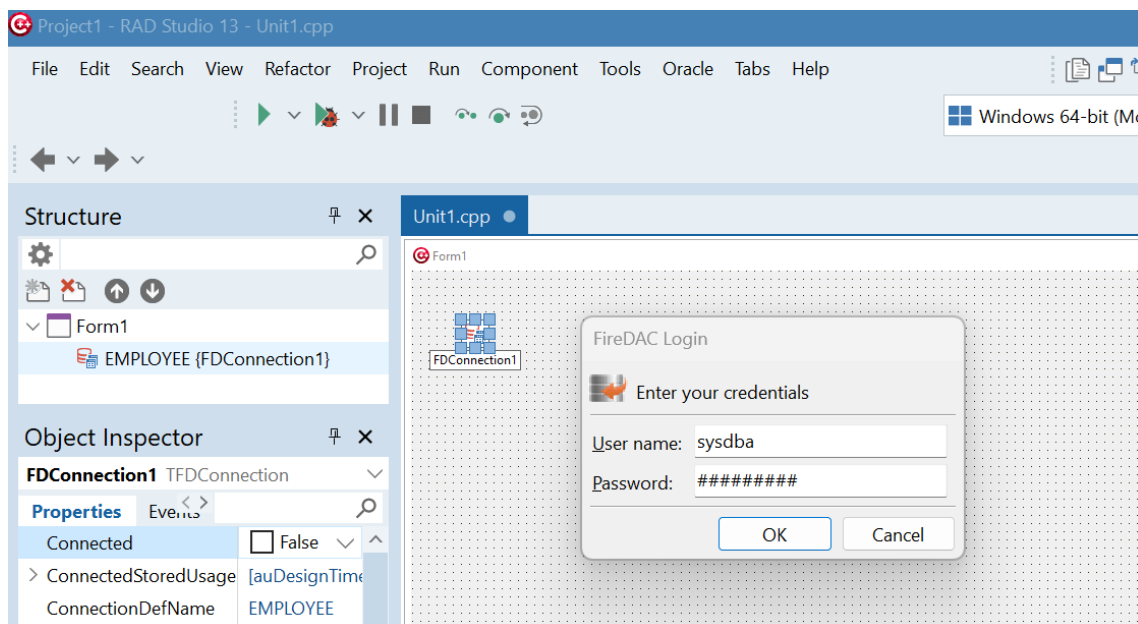
Vamos criar uma Nova Aplicação VCL e adicionar ao formulário de nossa aplicação um componente FDConnection da guia FireDAC da Tool Palette. Lembrando que este componente é responsável por estabelecer a conexão propriamente dita com o banco de dados.

Em seguida, com o FDConnection selecionado vamos ao Object Inspector (F11), e selecionar na lista suspensa da propriedade ConnectionDefName a opção EMPLOYEE



Utilizando essas predefinições não será necessário inserir nenhum parâmetro adicional (como nome do servidor ou banco de dados padrão) a nosso componente.

Depois de definir a propriedade Connected como **True**, o FireDAC exibirá uma caixa de diálogo de login:



Nela você deve inserir suas credenciais de usuário para acessar o banco de dados configurado. Por padrão, você tem direito a três tentativas, se todas falharem, o processo de login também falhará retornando uma mensagem de erro.

Pressione o botão OK para estabelecer a conexão efetivamente com o BD e consequentemente iniciar sua sessão de usuário no DBMS caso este suporte este recurso.

Caso a conexão tenha sido estabelecida com sucesso a propriedade Connected deve estar definida como True, caso contrário o valor da mesma será False e o FireDAC exibirá a mensagem de erro apropriada.

SELEÇÃO DE LINHAS DO BANCO DE DADOS USANDO C++BUILDER

Agora solte um componente TFDQuery, também da guia "FireDAC" da Tool Palette, no formulário. Este componente é responsável pela execução de comandos SQL que utilizaremos para buscar registros do banco de dados e enviar os dados alterados de volta ao banco de dados. Defina sua propriedade Connection como FDConnection1 para ligar a consulta a uma conexão de banco de dados.

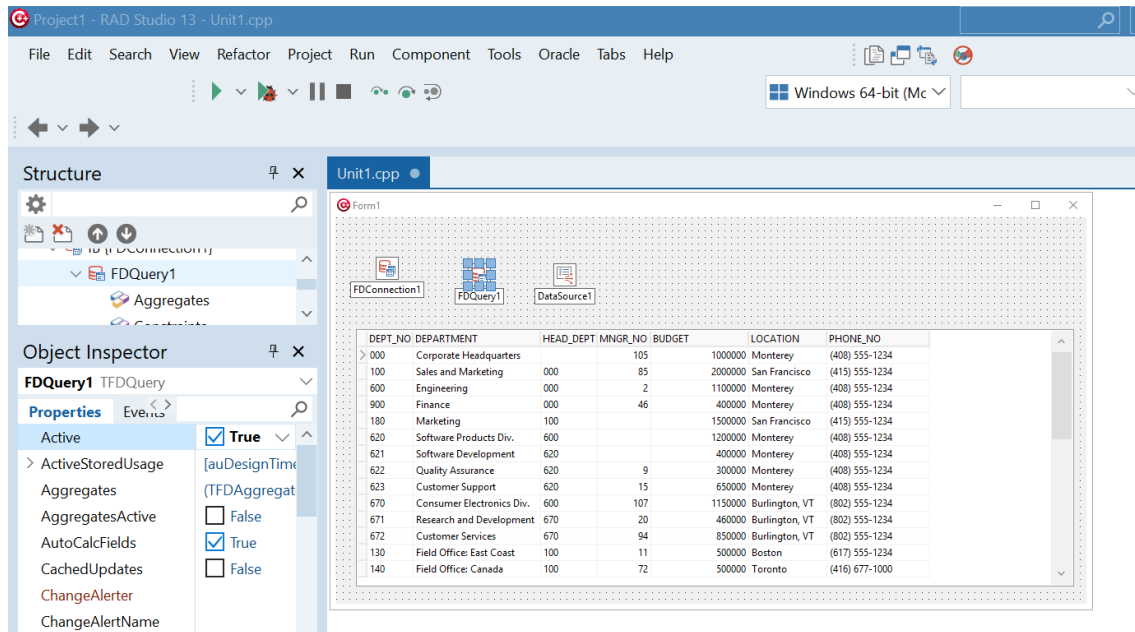
Nota: Se um componente de consulta (FDQuery) for adicionado em um formulário ou módulo de dados (DataModule) que já contenha um ou mais TFDConnections, o FireDAC define automaticamente a propriedade Connection da consulta para apontar para a conexão que foi criada primeiro.

Com o FDQuery selecionando, clique em sua propriedade SQL e insira o seguinte comando SQL na janela do editor:

```
SELECT * FROM DEPARTMENT
```

Pressione o botão OK para fechar o editor. Isso armazena o texto do comando SQL na propriedade SQL do componente TFDQuery. Em seguida, solte um componente C++Builder TDataSource padrão da guia "Data Access" da Tool Palette em seu formulário. Defina sua propriedade DataSet como FDQuery1. Agora coloque um controle TDBGrid no formulário da guia "Data Controls" e defina sua propriedade DataSource como DataSource1.

Por fim, defina a propriedade Active de FDQuery1 como True. Isso envia o comando SQL para o DBMS, que executa o comando e retorna um conjunto de resultados. Esses dados são exibidos pelo controle DBGrid1:



PREPARANDO O APLICATIVO PARA O TEMPO DE EXECUÇÃO C++BUILDER

Para permitir que seu aplicativo funcione em tempo de execução, você precisa:

- Inserir o componente TFDPhysIBDriverLink da página da paleta "Links FireDAC"
- Inserir o componente TFDGUIXWaitCursor da página da paleta "FireDAC UI"

Agora seu aplicativo está pronto para ser executado. Esses componentes garantem que as unidades necessárias sejam vinculadas ao executável do aplicativo. Para aplicativos do mundo real, esses componentes são normalmente colocados em um módulo de dados principal.

DEFININDO CONEXÕES EM TEMPO DE EXECUÇÃO

Em linhas gerais o FireDAC suporta 3 tipos de definições de conexão:

Modelo	Descrição	Prós	Contras
Persistente	Tem um nome exclusivo, é gerenciado pelo FDManager e é armazenado em um arquivo de definição de conexão.	Pode ser definido uma vez e reutilizado em muitos aplicativos. Pode ser agrupado.	Os parâmetros (endereço do servidor, nome do banco de dados e assim por diante) são visíveis publicamente e podem ser alterados acidentalmente.

			O FDManager deve ser reativado ou o Delphi IDE deve ser reiniciado para tornar visível uma definição recém-adicionada em tempo de design.
Privado	Tem um nome exclusivo, é gerenciado pelo FDManager, mas NÃO é armazenado em um arquivo de definição de conexão.	Os parâmetros de definição de conexão não são visíveis "fora" do aplicativo. Pode ser agrupado.	O aplicativo precisa criar uma definição de conexão privada após a reinicialização de cada programa e não pode compartilhá-la com outros programas. Não pode ser criado em tempo de design.
Temporário	Não tem nome, não é armazenado em um arquivo de definição de conexão e não é gerenciado pelo FDManager.	A maneira mais simples de criar uma definição de conexão é preencher propriedade TFDConnection.Params . Pode ser criado em tempo de design usando o editor de componente TFDConnection.	Semelhante ao privado. Também não pode ser referenciado por nome e não pode ser agrupado.

ARQUIVO DE DEFINIÇÃO DE CONEXÃO

As definições de conexão persistentes são armazenadas em um arquivo externo - o arquivo de definição de conexão. Este arquivo possui o formato de arquivo de texto INI padrão. Ele pode ser editada por FDExplorer ou FDAdministrator utilitários à primeira, manualmente, ou por código. Por padrão, o arquivo é:

C:\Users\Public\Documents\Embarcadero\Studio\FireDAC\FDConnectionDefs.ini.

Nota: Se você adicionar uma nova definição de conexão persistente usando FDExplorer ou FDAdministrator enquanto o RAD Studio IDE estiver em execução, ela não será visível para o código de tempo de design do FireDAC. Para atualizar a lista de definições de conexão persistente, você precisa reativar o FDManager ou reiniciar o RAD Studio IDE.

Amostra de conteúdo deste arquivo:

[Oracle_Demo]

DriverID=Ora

Database=ORA_920_APP

User_Name=ADDemo

Password=a

MetaDefSchema=ADDemo

;MonitorBy=Remote

[MSSQL_Demo]

DriverID=MSSQL

Server=127.0.0.1

Database=Northwind

User_Name=sa

Password=

MetaDefSchema=dbo

MetaDefCatalog=Northwind

MonitorBy=Remote

Um aplicativo pode especificar um nome de arquivo de definição de conexão na propriedade `FDManager.ConnectionDefFileName`. O FireDAC procura um arquivo de definição de conexão nos seguintes locais:

- Se `ConnectionDefFileName` for especificado:
 - procure um nome de arquivo sem um caminho e, em seguida, procure-o na pasta EXE do aplicativo.
 - caso contrário, apenas use um nome de arquivo especificado.
- Se `ConnectionDefFileName` não for especificado:
 - procure **FDConnectionDefs.ini** em uma pasta EXE do aplicativo.
 - Se o arquivo acima não for encontrado, procure um arquivo especificado na chave de registro **HKCU \ Software \ Embarcadero \ FireDAC \ ConnectionDefFile** . Por padrão, é `C:\Users\Public\Documents\Embarcadero\Studio\FireDAC\FDConnectionDefs.ini`.

Observação: o FireDAC ignora o valor de `FDManager.ConnectionDefFileName` em tempo de design e procura um arquivo na pasta *RAD Studio Bin* ou conforme especificado no registro. Se o arquivo não for encontrado, uma exceção é levantada.

Se `FDManager.ConnectionDefFileAutoLoad` for `True` , um arquivo de definição de conexão será carregado automaticamente. Caso contrário, ele deve ser carregado explicitamente chamando o método `FDManager.LoadConnectionDefFile` antes do primeiro uso das definições de conexão. Por exemplo, antes de definir `TFDConnection.Connected` como `True`.

CRIAÇÃO DE UMA DEFINIÇÃO DE CONEXÃO PERSISTENTE

Uma definição de conexão persistente pode ser criada utilizando ferramentas como **FDExplorer** ou **FDAdministrator**. Também é possível criar essa definição diretamente por código usando o **FireDAC**.

O exemplo abaixo cria uma definição de conexão chamada **MSSQL_Connection**, contendo todos os parâmetros necessários para se conectar a um **Microsoft SQL Server local**, utilizando autenticação por **usuário e senha** (sem autenticação do sistema operacional).

A definição é marcada como **persistente**, fazendo com que seja salva no arquivo **fdconnectiondefs.ini**, permitindo reutilização futura sem necessidade de recriação.

Includes necessários no C++Builder

```
#include <FireDAC.Comp.Client.hpp>
#include <FireDAC.Stan.Def.hpp>
#include <FireDAC.Stan.Intf.hpp>

// Necessários para MSSQL e persistência da conexão
#include <FireDAC.Phys.MSSQL.hpp>
#include <FireDAC.Phys.MSSQLDef.hpp>
```

Nome da definição de conexão

```
const String cNameConnDef = "MSSQL_Connection";
```

Criação da definição de conexão persistente

```
void __fastcall TForm1::PersistentConnectionClick(TObject *Sender)
{
    IFDStanConnectionDef oDef;
    TFDPhysMSSQLConnectionDefParams *oParams;

    // Cria uma nova definição de conexão
    oDef = FDManager->ConnectionDefs->AddConnectionDef();
    oDef->Name = cNameConnDef;

    // Parâmetros específicos do driver MSSQL
    oParams = static_cast<TFDPhysMSSQLConnectionDefParams*>(oDef->Params);

    oParams->DriverID = "MSSQL";
```

```

oParams->Database = "Northwind";
oParams->UserName = ".....";
oParams->Password = ".....";
oParams->Server = "127.0.0.1";
oParams->OSAuthent = false; // false = usuário/senha
oParams->MARS = false;

// Marca a definição como persistente
oDef->MarkPersistent();

// Salva a definição no arquivo fdconnectiondefs.ini
oDef->Apply();
}

```

Observações importantes (C++Builder)

- **FDManager** é uma instância global do gerenciador de conexões do FireDAC.
- **FDManager->ConnectionDefs** é uma coleção de definições de conexão.
- **AddConnectionDef()** cria uma nova definição.
- **MarkPersistent()** indica que a definição deve ser salva em disco.
- **Apply()** grava a definição no arquivo fdconnectiondefs.ini.
- Se **MarkPersistent()** não for chamado, a definição será **temporária** (válida apenas em tempo de execução).

CRIANDO UMA DEFINIÇÃO DE CONEXÃO PRIVADA

Uma definição de conexão privada é criada **exclusivamente por código** e existe apenas durante a execução da aplicação.

Diferente de uma definição persistente, **não é salva** no arquivo fdconnectiondefs.ini.

A criação é semelhante à definição persistente, porém **não utiliza MarkPersistent()** nem Apply().

Os parâmetros da conexão são informados através de uma lista de strings, onde cada item segue o formato Nome=Valor.

Includes necessários:

```

#include <System.Classes.hpp>
#include <FireDAC.Comp.Client.hpp>

```

Criando a definição de conexão privada:

```
void __fastcall TForm1::PrivateConnectionClick(TObject *Sender)
{
    TStrings *oParams;

    oParams = new TStringList();
    try
    {
        oParams->Add("Server=127.0.0.1");
        oParams->Add("Database=Northwind");
        oParams->Add("OSAuthent=Yes");

        // Cria a definição de conexão privada (somente em memória)
        FDManager->AddConnectionDef(
            "MSSQL_Connection", // Nome da definição
            "MSSQL",           // Driver
            oParams             // Parâmetros
        );
    }
    __finally
    {
        delete oParams;
    }
}
```

Utilizando a definição privada para conectar:

```
void __fastcall TForm1::ConnectionClick(TObject *Sender)
{
    FDConnection1->ConnectionDefName = "MSSQL_Connection";
    FDConnection1->Connected = true;
}
```

Observações importantes

- A definição criada com AddConnectionDef() **sem persistência**:
 - Não é gravada em disco
 - É válida apenas durante a execução da aplicação

- TStrings / TStringList é usado para informar os parâmetros no formato Chave=Valor
- Ideal para:
 - Conexões temporárias
 - Aplicações que não devem gravar configurações localmente
 - Ambientes de teste ou execução controlada

CRIAÇÃO DE UMA DEFINIÇÃO DE CONEXÃO TEMPORÁRIA

Uma definição de conexão temporária pode ser criada diretamente em tempo de execução preenchendo a propriedade **TFDConnection->Params**. Essa é a forma **mais simples e direta** de configurar uma conexão, pois não depende de definições persistentes nem privadas no FDManager.

Os parâmetros são adicionados no formato Chave=Valor e a conexão existe apenas enquanto o objeto TFDConnection estiver ativo.

Opção 1 – Usando Params

```
void __fastcall TForm1::TempConnectionParamsClick(TObject *Sender)
{
    FTDConnection1->Connected = false;
    FTDConnection1->Params->Clear();

    FTDConnection1->DriverName = "MSSQL";
    FTDConnection1->Params->Add("Server=127.0.0.1");
    FTDConnection1->Params->Add("Database=Northwind");
    FTDConnection1->Params->Add("User_Name=sa");

    FTDConnection1->Connected = true;
}
```

Opção 2 – UsandoConnectionString

Outra forma é definir todos os parâmetros em uma string de conexão. Essa abordagem é útil quando os dados de conexão vem de configuração externa (arquivo, banco, variável de ambiente, etc);

```
void __fastcall TForm1::TempConnectionStringClick(TObject *Sender)
{
    FTDConnection1->Connected = false;
```



```
FDConnection1->ConnectionString =  
    "DriverID=MSSQL;"  
    "Server=127.0.0.1;"  
    "Database=Northwind;"  
    "User_Name=sa";  
  
FDConnection1->Connected = true;  
}
```

Observações importantes

- A conexão é **temporária**:
 - Não é registrada no FDManager
 - Não é salva em fdconnectiondefs.ini
- Ideal para:
 - Aplicações simples
 - Scripts
 - Testes rápidos
 - Conexões dinâmicas
- Params → melhor quando os parâmetros são montados passo a passo
- ConnectionString → melhor quando tudo vem pronto em uma única string

EDITANDO UMA DEFINIÇÃO DE CONEXÃO

Uma aplicação pode precisar permitir que o usuário **crie ou edite uma definição de conexão em tempo de execução** utilizando o **Editor de Conexão padrão do FireDAC** (janela visual).

O FireDAC disponibiliza essa funcionalidade através da unidade **FireDAC.VCLUI.ConnEdit**, que abre o editor gráfico de conexão.

Includes necessários

```
#include <FireDAC.VCLUI.ConnEdit.hpp>  
#include <FireDAC.Comp.Client.hpp>
```

Editando uma conexão temporária armazenada em TFDConnection

Esse caso se aplica quando a conexão foi criada via Params ou ConnectionString diretamente no TFDConnection.

```
void __fastcall TForm1::EditConnectionClick(TObject *Sender)
{
    if (TfrmFDGUIxFormsConnEdit::Execute(FDConnection1, ""))
    {
        FDConnection1->Connected = true;
    }
}
```

O que acontece aqui:

- O editor visual do FireDAC é aberto
- O usuário altera servidor, banco, usuário etc.
- Se clicar em **OK**, a conexão é aplicada
- Se clicar em **Cancelar**, nada é alterado

Editando uma definição representada como string de conexão

Esse cenário é útil quando a conexão é manipulada como **string**, por exemplo:

- Configuração externa
- Armazenamento em arquivo ou banco
- Aplicações multi-ambiente

```
void __fastcall TForm1::EditConnectionStringClick(TObject *Sender)
{
    String sConnStr;

    // Obtém a string atual da definição de conexão
    sConnStr = FDConnection1->ResultConnectionDef->BuildString();

    // Abre o editor visual usando a string
    if (TfrmFDGUIxFormsConnEdit::Execute(sConnStr, ""))
    {
        // Aplica novamente a string editada
    }
}
```

```
FDConnection1->ResultConnectionDef->ParseString(sConnStr);  
  
FDConnection1->Connected = true;  
  
}  
  
}
```

Do Banco de Dados para a Tela (BD → UI)

1- Passagem codificada (clássica)

- Uso direto de componentes como:
 - TFDQuery
 - TDataSource
 - TDBEdit, TDBGrid, TDBComboBox

Fluxo:

TFDConnection → TFDQuery → TDataSource → Componente DB>

Exemplo:

```
FDQuery1->Open();
```

2- DBWare / DataSet tradicional

- Baseado em TDataSet
- Ideal para aplicações VCL clássicas
- Total controle manual do ciclo de vida dos dados

Vantagens:

- Simples
- Previsível
- Fácil debug

3- LiveBindings (abordagem moderna)

- Liga dados a controles **sem componentes DB visuais**
- Usa expressões e bindings declarativos

Vantagens:

- Menos código
- Interface mais desacoplada
- Ideal para formulários complexos

Exemplo de uso:

TFDQuery ↔ BindSourceDB ↔ LinkControlToField

Validações de Conexão

Testar conexão antes de usar:

```
try
{
    FDConnection1->Connected = true;
}
catch (Exception &e)
{
    ShowMessage("Erro ao conectar: " + e.Message);
}
```

Eventos importantes de TFDConnection

Antes de conectar...

```
void __fastcall TForm1::FDConnection1BeforeConnect(TObject *Sender)
{
    // Ajustes dinâmicos de parâmetros
}
```

Depois de conectar...

```
void __fastcall TForm1::FDConnection1AfterConnect(TObject *Sender)
{
    ShowMessage("Conectado com sucesso!");
}
```

Apurar erro:

```
void __fastcall TForm1::FDConnection1OnError(  
    TObject *Sender, Exception *E)  
{  
    ShowMessage("Erro de conexão: " + E->Message);  
}
```